

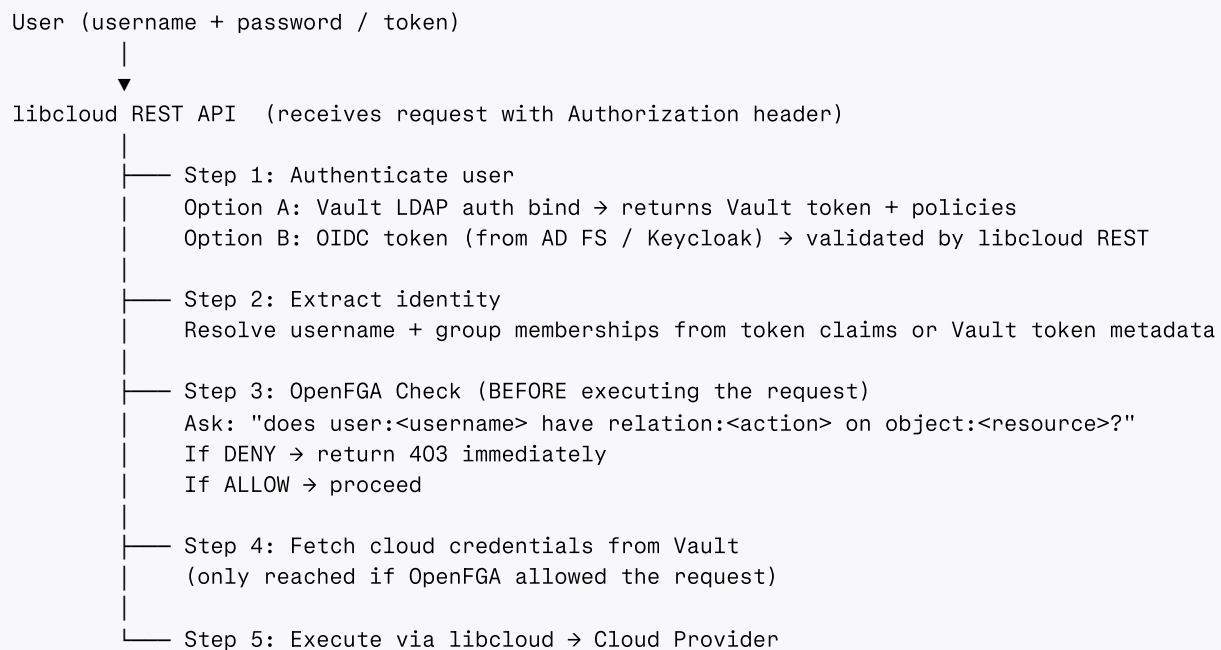


checking our design earlier, there is a OpenFGA component that authorized different level of roles to different path in the libcloud REST API. please fill in this part, as it is missing from above design.

Good catch. The missing piece is: how does a user's identity flow from AD/LLDAP through to OpenFGA, and what does the OpenFGA authorization model actually look like for libcloud REST API paths. Let me fill this in completely.

The Missing Auth Flow

The earlier architecture diagram skipped the step where libcloud REST gets the user's identity to pass to OpenFGA. Here is the complete flow:



OpenFGA sits as a **middleware authorization gate** inside libcloud REST — every inbound request is checked before any cloud operation is attempted.

OpenFGA Authorization Model for libcloud REST

Type Definitions

The model needs four types: `user`, `role`, `provider`, and `api_resource`.

```
model
  schema 1.1
```

```

type user

type role
  relations
    define member: [user]

type provider
  # e.g. object: provider:aws, provider:gcp, provider:azure, provider:nutanix
  relations
    define admin: [role#member]
    define operate: [role#member]
    define readonly: [role#member]

type api_resource
  # e.g. object: api_resource:compute#provision
  # Each HTTP method+path combination is an api_resource
  relations
    define can_execute: [role#member] or admin from provider or operate from provider or readonly from provider

```

In practice, the cleanest model maps **each logical API capability** (not raw HTTP path) to a relation on `api_resource`, with conditions inherited through the provider relationship. This avoids tuple explosion.

API Path → Relation Mapping

This is the core table that a developer must encode in the OpenFGA model. Each libcloud REST endpoint maps to a relation + resource object.

HTTP Method	libcloud REST Path	OpenFGA Object	OpenFGA Relation	Minimum Role
GET	/compute/{provider}/nodes	api:compute.list	can_read	readonly
GET	/compute/{provider}/nodes/{id}	api:compute.list	can_read	readonly
POST	/compute/{provider}/nodes	api:compute.provision	can_write	operator
POST	/compute/{provider}/nodes/{id}/action	api:compute.action	can_write	operator
DELETE	/compute/{provider}/nodes/{id}	api:compute.destroy	can_delete	admin
GET	/compute/{provider}/sizes	api:compute.meta	can_read	readonly
GET	/compute/{provider}/images	api:compute.meta	can_read	readonly
POST	/compute/{provider}/keypairs	api:compute.keypair	can_write	operator
DELETE	/compute/{provider}/keypairs/{id}	api:compute.keypair	can_delete	admin
GET	/storage/{provider}/buckets	api:storage.list	can_read	readonly
POST	/storage/{provider}/buckets	api:storage.bucket	can_write	operator
DELETE	/storage/{provider}/buckets/{id}	api:storage.bucket	can_delete	admin
PUT	/storage/{provider}/buckets/{id}/objects	api:storage.object	can_write	operator
GET	/storage/{provider}/buckets/{id}/objects	api:storage.object	can_read	readonly
GET	/network/{provider}/networks	api:network.list	can_read	readonly
POST	/network/{provider}/floatingips	api:network.floatingip	can_write	operator

HTTP Method	libcloud REST Path	OpenFGA Object	OpenFGA Relation	Minimum Role
DELETE	/network/{provider}/floatingips/{id}	api:network.floatingip	can_delete	admin
GET	/admin/users	api:admin.users	can_read	cloud-owner
POST	/admin/users	api:admin.users	can_write	cloud-owner
POST	/admin/vault/rotate	api:admin.vault	can_execute	cloud-owner

Role → Relation → API Access Matrix

```

cloud-owner
| can_read, can_write, can_delete, can_execute on ALL api_resource objects
| admin on ALL provider objects
|
cloud-admin
| can_read, can_write, can_delete on compute/storage/network api_resource objects
| operate on ALL provider objects (no admin-level destructive ops)
|
cloud-operator-{aws|gcp|azure|...} (provider-scoped)
| can_read, can_write on compute/storage/network for THAT provider only
| operate on provider:{aws} only
|
cloud-readonly-{aws|gcp|azure|...}
| can_read on ALL api_resource objects for THAT provider only
| readonly on provider:{aws} only

```

Provider-scoped roles are the key insight — a user can be operator on provider:aws but readonly on provider:gcp, and OpenFGA enforces this per-request without any application-level if/else logic.

Example Tuples Written at Role Assignment

When chain-role-assign.sh runs for a new cloud operator on AWS:

```

# user alice is a member of the cloud-operator-aws role
fga tuple write user:alice member role:cloud-operator-aws

# the cloud-operator-aws role has "operate" on provider:aws
fga tuple write role:cloud-operator-aws#member operate provider:aws

# (these provider→api_resource tuples are written once at system setup, not per user)
fga tuple write role:cloud-operator-aws#member can_read api:compute.list
fga tuple write role:cloud-operator-aws#member can_write api:compute.provision
fga tuple write role:cloud-operator-aws#member can_write api:compute.action
# ... etc for all operator-level resources

```

When alice calls POST /compute/aws/nodes, libcloud REST checks:

```

user:alice can_write api:compute.provision

```

OpenFGA traverses: `alice` → member of `role:cloud-operator-aws` → which has `can_write` on `api:compute.provision` → **ALLOW**.

Where This Fits in the Script Inventory

Two scripts from the earlier inventory now have concrete implementations:

```
- openfga-tuple-reconcile.py – when it syncs LDAP/AD group memberships, it writes the `us
```

```
- chain-role-assign.sh – after adding the user to the LDAP group, it calls openfga-tuple
```

This design keeps the tuple count at $O(\text{users} \times \text{roles})$ rather than $O(\text{users} \times \text{API paths})$, which is the correct OpenFGA pattern for RBAC at scale. ^[1] ^[2]



1. <https://openfga.dev/docs/concepts>

2. <https://openfga.dev/docs/interacting/managing-relationships-between-objects>